Tesina Progetto Sistemi Operativi A.A 24-25

Servizio di Messaggistica Client–Server TCP Multithreading

Alessio Pompeo - 0310289

1. Specifiche progetto – Servizio di messaggistica:

Realizzazione di un servizio di scambio messaggi supportato tramite un server sequenziale o concorrente (a scelta). Il servizio deve accettare messaggi provenienti da client (ospitati in generale su macchine distinte da quella dove risiede il server) ed archiviarli.

L'applicazione client deve fornire ad un utente le seguenti funzioni:
1. Lettura tutti i messaggi spediti all'utente.
2. Spedizione di un nuovo messaggio a uno qualunque degli utenti del sistema.
3. Cancellare dei messaggi ricevuti dall'utente.

Un messaggio deve contenere almeno i campi Destinatario, Oggetto e Testo.

Si precisa che la specifica prevede la realizzazione sia dell'applicazione client che di quella server. Inoltre, il servizio potra' essere utilizzato solo da utenti autorizzati (deve essere quindi previsto un meccanismo di autenticazione).

2. Architettura di sistema:

2.1. Overview:

L'architettura segue il classico schema **client-server asimmetrico su rete TCP/IP**. Il server è un processo permanente in ascolto sulla **porta 9090** che accetta connessioni da client arbitrari, anche su macchine fisicamente distinte, come previsto dalla specifica. Ogni client stabilisce una connessione TCP persistente, si autentica e poi interagisce con il server tramite una sequenza di richieste e risposte fino alla chiusura esplicita della sessione.

La comunicazione avviene attraverso lo scambio di strutture dati di tipo **Packet**, trasmesse come blocchi binari di dimensione fissa pari a **sizeof(Packet)**. Questo approccio elimina la necessità di un layer di serializzazione e garantisce che ogni operazione di **send** o **recv** trasferisca esattamente un'unità logica di dato. Il campo type all'interno di ogni pacchetto funge da discriminante dell'operazione, permettendo al server di smistare le richieste verso il gestore appropriato con un semplice costrutto switch.

Il server mantiene tutta la persistenza su due file di testo nella directory di lavoro: **users.txt** per le credenziali degli utenti e **messages.txt** per i messaggi archiviati. La scelta di un formato testuale leggibile facilita l'ispezione manuale durante lo sviluppo e il debug.

2.1.1 Componenti principali

Il progetto è composto dai seguenti file:

- **server.c**: *Implementa il server di messaggistica.*
- **client.c**: *Implementa l'applicazione client con interfaccia testuale da terminale Linux.*
- **common.h**: *Contiene le strutture dati e le costanti condivise tra client e server.*

Scelta: Server Concorrente basato su Thread

Il server utilizza un modello concorrente con **thread POSIX (pthread)**. Ogni client connesso viene gestito da un thread separato, permettendo:

- Elaborazione parallela di più richieste simultanee
- Maggiore reattività nel servire più client
- Sincronizzazione tramite mutex per proteggere l'accesso alle risorse condivise (file degli utenti, file dei messaggi)

Alternativa scartata: Server sequenziale

- Avrebbe causato blocchi prolungati durante l'accesso ai file.
- Performance ridotte per una mancanza di uso delle complete risorse di sistema thread.
- Peggiore gestione del codice

2.2. Struttura dei messaggi sender/receiver:

La comunicazione tra client e server avviene tramite una struttura dati comune:

```
typedef struct {
    PacketType type;
    char sender[MAX_USER];
    char receiver[MAX_USER];
    char subject[MAX_OBJ];
    char body[MAX_BODY];
    time_t timestamp;
    int id;  // id messaggio per cancellazione
} Packet;
```

Questa struttura rappresenta un **pacchetto di comunicazione** e contiene:

- **PacketType**: tipo di richiesta
- **Sender**: utente mittente
- **Receiver**: utente destinatario
- **Subject**: oggetto del messaggio
- **Body**: contenuto del messaggio
- **Timestamp**: data e ora
- **Id**: identificativo univoco

L'unità di scambio tra client e server è la **struct Packet**, definita in **common.h**. Ogni pacchetto trasporta un campo type (di tipo enumerato PacketType) che identifica l'operazione, più i campi dati necessari**: sender, receiver, subject, body, timestamp e id.**

Il protocollo adotta un approccio a lunghezza fissa: ogni scambio consiste in esattamente un send e un recv di sizeof(Packet) byte. Questo elimina la complessità del framing (delimitatori, lunghezza prefissa) e rende il codice di invio e ricezione estremamente semplice e uniforme. Il compromesso è una certa ridondanza: campi non utilizzati in alcuni tipi di pacchetto vengono comunque trasmessi, aumentando il traffico di rete rispetto a un protocollo compatto con serializzazione variabile.

Il campo subject viene riutilizzato come flag di controllo in diversi contesti: vale 'LOGIN' o 'REGISTER' nel pacchetto di autenticazione, 'AUTH_OK' o 'NO_RECEIVER' nella risposta CHECK_USER, 'OK' / 'NOT_FOUND' / 'ERR_IO' nella risposta di cancellazione. Analogamente, il campo body trasporta la password nel pacchetto AUTH_REQUEST. Questo multiplexing riduce il numero di tipi di pacchetto necessari a scapito di una minore chiarezza semantica.

2.3. Operazioni pacchetto supportate:

```
typedef enum {
    AUTH_REQUEST,
    AUTH_SUCCESS,
    AUTH_FAIL,
    SEND_MESSAGE,
    READ_MESSAGES,
    CHECK_USER,
    DELETE_MESSAGE,
    MESSAGE_LIST,
    END_SESSION
} PacketType;
```

Il protocollo definisce diversi tipi di pacchetto:

- **AUTH_REQUEST**: richiesta di autenticazione
- **AUTH_SUCCESS**: autenticazione riuscita
- **AUTH_FAIL**: autenticazione fallita
- **SEND_MESSAGE**: invio di un nuovo messaggio
- **READ_MESSAGES**: richiesta di lettura dei messaggi
- **DELETE_MESSAGE**: richiesta di cancellazione di un messaggio
- **END_SESSION**: chiusura della sessione client

2.4 Meccanismo di Autenticazione

Il servizio è accessibile esclusivamente a utenti autenticati, come richiesto dalla specifica. Il flusso di autenticazione si articola come segue: il client invia un pacchetto AUTH_REQUEST con username nel campo sender e password nel campo body; il campo subject vale 'LOGIN' per una richiesta di accesso o 'REGISTER' per una nuova registrazione. Il server risponde con AUTH_SUCCESS o AUTH_FAIL.

La funzione register_user verifica preventivamente l'assenza dello username nel file prima di aggiungere il nuovo utente, garantendo l'unicità degli identificatori. La funzione authenticate_user confronta le credenziali ricevute con quelle archiviate riga per riga. Entrambe le operazioni sono protette da mutex distinti per minimizzare la contesa in presenza di accessi concorrenti.

Si riconosce che la memorizzazione delle password in chiaro costituisce una limitazione rispetto a standard di sicurezza moderni.

2.5 Generazione degli identificatori di messaggio

Ogni messaggio è identificato da un intero generato lato client dalla funzione **generate_message_id(),** che combina il timestamp corrente con un valore pseudocasuale tramite **XOR** e riduzione modulo **10^6**. Il seed di rand() è inizializzato nel main con una combinazione di **time(NULL), getpid()** e l'indirizzo di printf, per aumentare l'entropia rispetto al solo timestamp.

La generazione lato client semplifica il flusso di comunicazione: il client conosce l'ID del messaggio prima di inviarlo, permettendo di stampare un messaggio di conferma immediato senza attendere una risposta del server contenente l'ID assegnato. Il server non valida l'unicità degli ID ricevuti, il che rappresenta una semplificazione consapevole adatta al contesto didattico.

2.6 Gestione dei segnali UNIX

Il **server** intercetta **SIGINT (Ctrl+C)** tramite un handler dedicato che imposta a zero la variabile globale volatile **sig_atomic_t server_running e** chiude il socket server. Questo causa il ritorno di accept() con errore EBADF, permettendo al loop principale di rilevare la condizione di uscita e terminare in modo pulito con un messaggio di log. Tutti gli altri segnali vengono esplicitamente ripristinati ai valori di default all'avvio, evitando di ereditare comportamenti imprevisti dall'ambiente di lancio. SIGTSTP è ignorato per impedire la sospensione accidentale del server da terminale.

Il **client** adotta una strategia diversa: ignora sia SIGINT che SIGTSTP per evitare interruzioni durante l'inserimento interattivo di dati, una situazione che potrebbe lasciare il socket in uno stato inconsistente. L'utente termina la sessione esclusivamente tramite l'opzione 4 del menu, che invia un pacchetto END_SESSION al server prima di chiudere il socket.

2.7. Diagrammi sequenze Client-Server:

2.7.1 Autenticazione utente

Il flusso di autenticazione è il seguente:

```
Client                        Server
  |                             |
  |------ AUTH_REQUEST ------->|
  |                             |
  |<----- AUTH_SUCCESS --------|
  |            oppure           |
  |<----- AUTH_FAIL -----------|
```

Se l’autenticazione ha successo, il client può accedere alle funzionalità del servizio.

2.7.2 Invio di un messaggio

```
Client                        Server
  |                             |
  |------ SEND_MESSAGE ---------->|
  |                             |
  |---- verifica destinatario ----|
  |                             |
  |<------ CHECK_USER ------------|
  |                             |
  |---- salvataggio messaggio ----|
```

Il server:

1. controlla l’esistenza del destinatario
2. assegna timestamp e ID
3. salva il messaggio nel file dei messaggi.

2.7.3 Lettura dei messaggi

```
Client                        Server
  |                             |
  |---- READ_MESSAGES ---------->|
  |                             |
  |<---- MESSAGE_LIST -----------|
  |<---- MESSAGE_LIST -----------|
  |<---- MESSAGE_LIST -----------|
```

Il server invia al client tutti i messaggi destinati all’utente.

2.7.4 Cancellazione di un messaggio

```
Client                          Server
  |                                |
  |---- DELETE_MESSAGE ---------->|
  |                                |
  |---- scansione file ----------|
  |                                |
  |<------ OK / NOT_FOUND -------|
```

Il server verifica che il messaggio appartenga all'utente prima della cancellazione.

2.7.5 Flusso Operazioni principali

Autenticazione: Client → AUTH_REQUEST (type, sender=user, body=pass, subject='LOGIN'|'REGISTER') → Server → AUTH_SUCCESS | AUTH_FAIL.

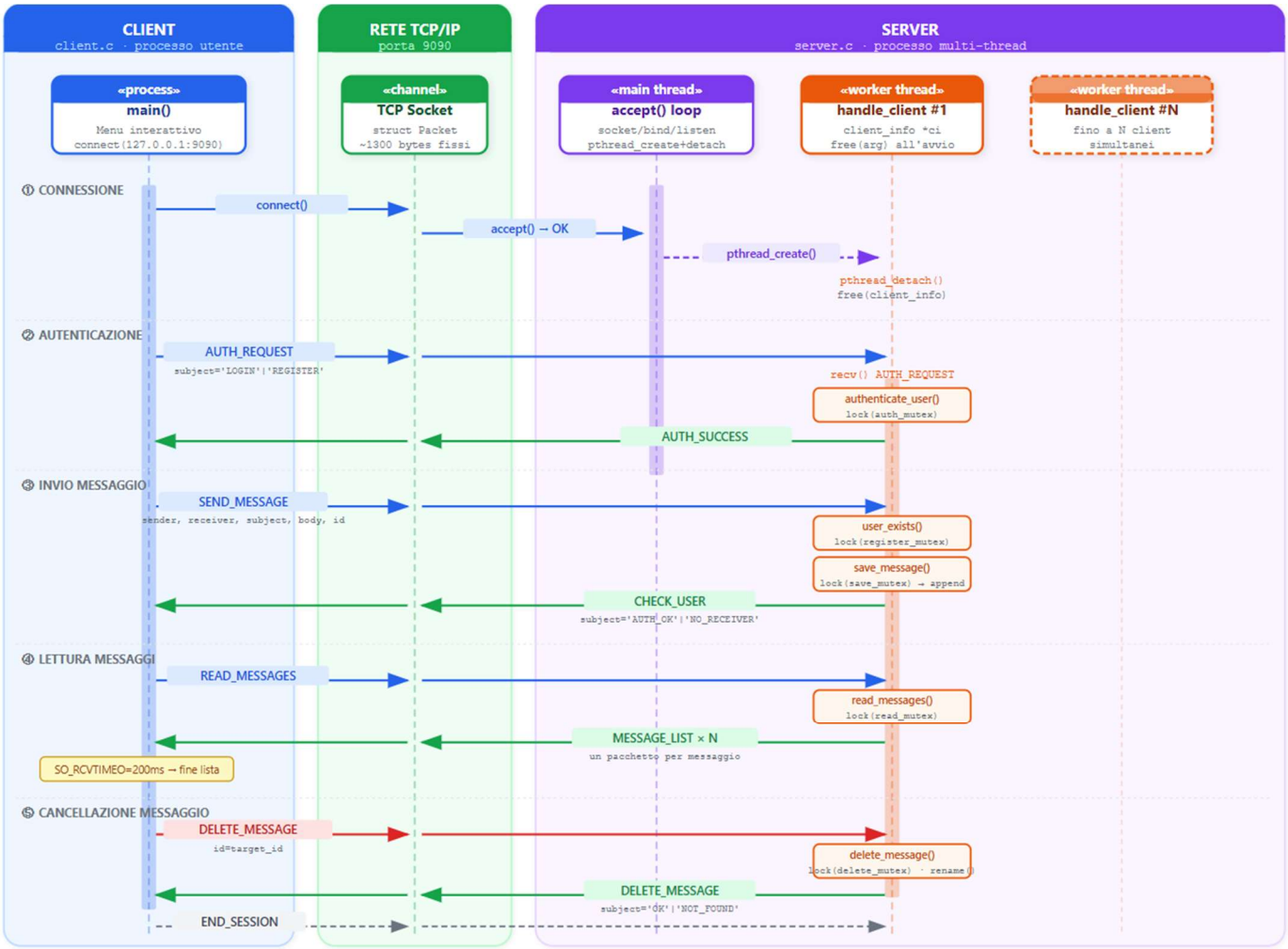
Invio messaggio: Client → SEND_MESSAGE (tutti i campi compilati) → Server → CHECK_USER (subject='AUTH_OK'|'NO_RECEIVER').

Lettura messaggi: Client → READ_MESSAGES → Server invia N × MESSAGE_LIST → timeout 200ms termina la ricezione lato client.

Cancellazione: Client → DELETE_MESSAGE (id=target_id) → Server → DELETE_MESSAGE (subject='OK'|'NOT_FOUND'|'ERR_IO').

Fine sessione: Client → END_SESSION → Server chiude socket e termina il thread.

2.7.6 Design architetturale



3. Scelte di progetto:

3.1. Modello di server concorrente:

Il server è stato progettato come **server concorrente multithread**.

Per ogni nuova connessione Server-Client:

1. il server main thread esegue **accept()**
2. viene creato un nuovo server thread child tramite **pthread_create()**
3. il **server thread child** gestisce la sessione del client

Questo approccio consente:

- gestione simultanea di più client
- maggiore scalabilità
- riduzione dei tempi di attesa tra utenti

Ogni thread esegue la funzione: **handle_client()** che gestisce tutte le operazioni del client fino alla disconnessione.

3.2. Comunicazione tramite socket TCP:

La comunicazione avviene tramite **socket TCP**.

Il server:

- crea un socket con **socket()**
- associa la porta con **bind()**
- si mette in ascolto con **listen()**
- accetta connessioni con **accept()**

Il client:

- crea il **socket**
- si connette al server tramite **connect()**

Lo scambio dati avviene tramite: **send()** e **recv()**

Il protocollo è **basato su strutture Packet serializzate direttamente**.

3.3. Persistenza dati:

Scelta: **File-based Storage**

I **dati** sono archiviati in file di testo:

- **users.txt**: Credenziali utenti (username password)
- **messages.txt**: Messaggi archiviati (formato: timestamp|id|sender|receiver|subject|body)

Vantaggi:

- Semplicità di implementazione
- Facilità di debugging e ispezione manuale
- Adatto a un progetto didattico

Limitazioni:

- Non scalabile per milioni di messaggi
- Poco sicuro
- Corruzione e perdita dei dati possibile

4. Gestione degli utenti:

4.1 Registrazione:

```
#define USERS_FILE "users.txt"
```

Gli utenti sono memorizzati nel file: **users.txt**

Il formato è: **username password**

```
mario 1234
anna qwerty
luca password
```

Durante la registrazione il server:

1. verifica se l'utente esiste già
2. se non esiste, aggiunge la nuova coppia username/password al file.

Questa operazione è protetta da **mutex (register_mutex)** per evitare race conditions tra thread.

4.2 Autenticazione:

Durante il login il server:

1. apre il file degli utenti
2. confronta username e password
3. invia al client uno dei seguenti risultati: AUTH_SUCCESS oppure AUTH_FAIL. Solo gli utenti autenticati possono utilizzare il servizio.
4. Esegue controlli su ridondanza degli utenti

4.3 Gestione dei messaggi:

I messaggi sono memorizzati nel file: **messages.txt**

```
#define MSG_FILE "messages.txt"
```

Il formato utilizzato è: timestamp|id|sender|receiver|subject|body

```
if(fprintf(fp, "%ld|%d|%s|%s|%s|%s\n",
  (long)pkt->timestamp,
        modified_id,
        pkt->sender,
        pkt->receiver,
        pkt->subject,
        pkt->body) < 0)
```

Questo formato permette una semplice gestione tramite lettura sequenziale del file.

```
1764351793|780129|alessio|anna|chat|ciao
1764351822|72576|anna|alessio|chat|ciao
```

4.4 Invio dei messaggi:

Quando un client invia un messaggio:

1. il server verifica che il destinatario esista
2. assegna un timestamp
3. salva il messaggio nel file

Il messaggio viene scritto con: **fprintf()**

L'accesso concorrente al file è protetto da: **save_mutex**

4.5 Lettura dei messaggi:

Per la lettura dei messaggi:

1. il server scorre il file messages.txt
2. seleziona i messaggi destinati all'utente
3. li invia al client

Il client visualizza ogni messaggio nel formato: **(ID) [timestamp]**

Da: mittente | Oggetto: oggetto | Testo: contenuto

4.6 Cancellazione dei messaggi:

La cancellazione di un messaggio avviene tramite:

1. lettura del file messages.txt
2. scrittura dei messaggi rimanenti in un file temporaneo
3. sostituzione del file originale

File utilizzato: **messages_tmp.txt**

Se il messaggio esiste e appartiene all'utente:

- viene eliminato
- il server invia risposta OK

altrimenti:

- viene restituito NOT_FOUND.

5. Sincronizzazione tra threads:

Poiché il server è multithread, è necessario evitare **race condition** sull'accesso ai file.

Questo garantisce **consistenza dei dati anche con più client attivi contemporaneamente**.

Sono stati utilizzati **mutex POSIX**:

Mutex	Funzione
register_mutex	gestione registrazione
auth_mutex	autenticazione
file_mutex	oper. su file
stdin_mutex	lettura messaggi

6.Gestione dei segnali:

Il server gestisce il segnale: SIGINT per permettere una **chiusura pulita del servizio**.

Quando il segnale viene ricevuto:

1. viene impostata la variabile **server_running = 0**
2. il socket del server viene chiuso
3. il ciclo principale termina.

Questo evita la terminazione brusca del server.

```
volatile sig_atomic_t server_running = 1;
```

```
struct sigaction sa;
sa.sa_handler = sigint_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);
```

```
void sigint_handler(int signo) {
    (void)signo;
    printf("\n[SERVER] Ricevuto SIGINT, chiusura in corso...\n");
    server_running = 0;
    close(server_sock);
}
```

```
while (server_running) {
    client_info *ci = malloc(sizeof(client_info));
    if (!ci) {
        fprintf(stderr, "malloc failed\n");
        continue;
    }

    socklen_t len = sizeof(ci->addr);
    ci->sock = accept(server_sock, (struct sockaddr*)&ci->addr, &len);
    if (ci->sock < 0) {
        free(ci);
        if (!server_running) break; // uscita pulita
        fprintf(stderr, "accept failed\n");
        continue;
    }

    pthread_t tid;
    if (pthread_create(&tid, NULL, handle_client, ci) != 0) {
        fprintf(stderr, "pthread_create failed\n");
        close(ci->sock);
        free(ci);
        continue;
    }
    pthread_detach(tid);
}

printf("[SERVER] Chiusura completata.\n");
close(server_sock);
```

7.Robustezza del sistema:

Per migliorare l'affidabilità del programma sono state adottate alcune tecniche:

7.1 gestione degli errori

Tutte le principali system call verificano il valore di ritorno, permettendo al programma di uscire in errore o di continuare l'esecuzione anche in casistiche non convenzionali.

7.2 gestione di EINTR, I/O e retry

Le chiamate di sistema di I/O su file (fscanf, fprintf, fgets) e su socket (scanf nel client) possono essere interrotte da segnali, restituendo errore con errno impostato a EINTR. Il codice gestisce esplicitamente questo caso tramite etichette goto di retry (ad esempio read_again:, write_again:) che ripetono l'operazione interrotta. Questa tecnica, standard nella programmazione POSIX, garantisce che l'arrivo di un segnale durante un'operazione di I/O non causi la perdita silenziosa di dati o la terminazione prematura di un ciclo di lettura.

7.3 strategia cancellazione messaggi

La cancellazione di un messaggio non può avvenire in-place su un file di testo, poiché i file POSIX non supportano la cancellazione arbitraria di porzioni interne senza riscrivere l'intero contenuto. La funzione delete_message() implementa il pattern *read-filter-write*: apre messages.txt in lettura e il file temporaneo messages_tmp.txt in scrittura, copia tutte le righe ad eccezione di quella con l'id e il destinatario cercati, poi chiude entrambi i file e invoca rename() per sostituire atomicamente il file originale con quello filtrato.

La chiamata rename() è atomica a livello di filesystem POSIX quando source e destination risiedono sullo stesso filesystem: questo garantisce che il file dei messaggi non sia mai in uno stato intermedio parzialmente scritto, anche in caso di crash del processo tra la fase di scrittura del temporaneo e la fase di rinomina.

7.4 persistenza TCP e timeout

Il client stabilisce una singola connessione TCP all'avvio e la mantiene attiva per tutta la sessione. Questo approccio, detto connection-per-session, evita il costo di three-way handshake ripetuto per ogni operazione e permette al server di mantenere lo stato dell'utente autenticato associato al socket per tutta la durata della sessione, senza necessità di trasmettere un token di sessione ad ogni richiesta.

La lettura dei messaggi utilizza un meccanismo di timeout per rilevare la fine della lista: il client imposta SO_RCVTIMEO a 200 ms prima di entrare nel loop di ricezione e interpreta il timeout come segnale che il server ha terminato l'invio. Il server non invia alcun pacchetto sentinella esplicito. Questa scelta semplifica il codice server (nessuna logica di terminazione lista) al costo di una lieve latenza aggiuntiva alla fine di ogni lettura e di una fragilità potenziale in reti ad alta latenza.

7.5 Persistenza su file di testo

La persistenza dei dati è realizzata tramite due file di testo nella directory di lavoro del server. Il file users.txt memorizza una coppia username-password per riga, separati da spazio. Il file messages.txt archivia un messaggio per riga nel formato:

```
timestamp|id|sender|receiver|subject|body
```

La scelta del formato testuale con separatore pipe è motivata dalla semplicità di parsing tramite sscanf con specificatori %[^|], dalla leggibilità immediata durante il debug e dalla portabilità assoluta tra sistemi. Il separatore pipe è stato preferito alla virgola per ridurre i conflitti con testi in lingua italiana che usano frequentemente la virgola. Una limitazione di questo approccio è che il carattere pipe nei campi dati corromperebbe il parsing: non è prevista alcuna forma di escaping.

8. Tecniche e Metodologie Utilizzate

8.1 Programmazione Concorrente (POSIX Threads)

- pthread_create(): Creazione di un thread per ogni client
- pthread_detach(): Liberazione automatica delle risorse del thread
- pthread_mutex_lock() / pthread_mutex_unlock(): Sezioni critiche protette
- volatile sig_atomic_t: Variabile di controllo sicura per i segnali

8.2 Gestione delle Risorse

- malloc() / free(): Allocazione dinamica per strutture client_info
- fopen() / fclose(): Gestione file con protezione mutex
- close(): Chiusura socket in caso di errore o disconnessione

8.3 Gestione dei Segnali (Signal Handling)

- sigaction(): Registrazione di handler robusti
- sigset_t / sigemptyset(): Manipolazione dei set di segnali
- SIG_IGN / SIG_DFL: Ignorare o usare comportamento default

8.4 Programmazione Socket (TCP/IP)

- socket() / bind() / listen(): Setup server
- accept(): Accettazione connessioni client
- connect(): Connessione lato client
- send() / recv(): Comunicazione bidirezionale
- SO_REUSEADDR: Permette riavvio rapido del server

8.5 Parsing e Serializzazione Dati

- sscanf(): Parsing formato testuale dei messaggi archiviati
- sprintf() / fprintf(): Serializzazione in formato persistente
- strncpy(): Copia sicura con limite di lunghezza

8.6 Acquisizione messaggi

- fgets: acquisizione linee da stdin
- scanf/fscanf: acquisizione stringhe da stdin

9. Struttura del codice

Il codice è organizzato in tre file con responsabilità ben distinte. common.h è incluso da entrambi i file sorgente e definisce il contratto di comunicazione: tutte le costanti dimensionali, l'enumerazione PacketType e la struct Packet. Qualunque modifica al protocollo richiede la modifica di questo solo file, mantenendo la coerenza tra client e server.

In server.c tutte le funzioni operative sono dichiarate static, limitando la loro visibilità al file di compilazione e riducendo la probabilità di conflitti di nomi. La funzione handle_client costituisce il corpo del thread worker ed è l'unica ad accedere al socket del client: riceve il pacchetto di autenticazione, poi entra in un loop di dispatching basato sul campo type. Le funzioni di persistenza **(register_user, authenticate_user, user_exists, save_message, read_messages, delete_message)** sono autonome e incapsulano completamente la gestione dei rispettivi mutex.

In client.c la logica di interazione con l'utente è contenuta interamente in main(), strutturata in una fase di autenticazione seguita da un loop di menu. Le funzioni **print_packet e generate_message**_id sono funzioni pure di utilità senza effetti collaterali globali, facilmente testabili in isolamento.

10. Limitazioni del sistema

Il sistema presenta alcune limitazioni:

- i dati sono salvati su file e non su database
- non esiste cifratura delle password, ogni carattere risulta totalmente in chiaro sul file.
- non è implementato un sistema di notifiche in tempo reale che renderebbe il sistema più responsive.
- la lettura dei messaggi avviene tramite scansione sequenziale del file che può risultare lenta in contesti con migliaia di dati.
- Possibili errori di parsing di messaggi e utenti se viene immesso il carattere | oppure \\ sono stati tamponati usando le funzioni **contains_forbidden** e **fgets_validated**, che permettono con opportune implementazioni nel codice di evitare che l'utente possa manomettere il sistema accidentalmente o non. Una soluzione maggiormente oculata sarebbe quella di costruire funzioni apposite per rilevare l'ingresso di questi caratteri senza prevenirne l'uso.

11. Manuale d'uso:

11.1 Requisiti di sistema

Il progetto richiede un sistema operativo GNU/Linux (o compatibile POSIX) con GCC installato (versione 7 o superiore per il supporto completo a C17) e la libreria pthreads disponibile (presente di default in tutte le distribuzioni Linux). Non sono necessarie dipendenze aggiuntive.

11. 2 Compilazione

Posizionarsi nella directory contenente i tre file sorgente ed eseguire i seguenti comandi:

```
# Compilazione del server (richiede -lpthread per POSIX threads)
gcc server.c -o server -lpthread
# Compilazione del client
gcc client.c -o client
```

Per abilitare tutti i warning e le ottimizzazioni di debug è consigliato aggiungere i flag -Wall -Wextra -g:

```
gcc server.c -o server -lpthread -Wall -Wextra -g
gcc client.c -o client -Wall -Wextra -g
```

11.3 Avvio del sistema

Il server deve essere avviato prima di qualsiasi client. Aprire due terminali separati nella stessa directory dei file eseguibili:

```
# Terminale 1 — avvio del server
./server
# Output atteso:
# Server in ascolto sulla porta 9090...
# Terminale 2 — avvio del client
./client
```

Il server rimane in esecuzione fino alla ricezione di SIGINT (Ctrl+C). Il client termina quando l'utente seleziona l'opzione 4 (Esci) dal menu principale.

11.4 Connessione da macchine remote

Per connettersi a un server in esecuzione su una macchina remota è necessario modificare l'indirizzo IP nel client da "127.0.0.1" all'indirizzo IP della macchina server. La porta 9090 deve essere raggiungibile dalla macchina client (verificare le regole del firewall se necessario).

11.5 Gestione di istanze precedenti

Se il server non è stato terminato correttamente e la porta 9090 risulta ancora occupata, è possibile liberarla con uno dei seguenti comandi:

```
sudo lsof -t -i tcp:9090 | xargs sudo kill -9
# oppure
sudo fuser -k 9090/tcp
```

12. Scenari d'uso passo passo

12.1 Registrazione di un nuovo utente

All'avvio il client presenta il menu di accesso. Selezionare l'opzione 2 (Registrazione), inserire username e password desiderati. Il server verifica l'assenza dello username nel file users.txt e, se non esiste, aggiunge le credenziali e risponde con AUTH_SUCCESS. Il client conferma l'avvenuta registrazione e prosegue con il menu principale.

```
1. Login
2. Registrazione
Scelta:
```

```
1. Login
2. Registrazione
Scelta: 2
Username: alessio
Password: prova
Utente già presente, scegli un altro username.
Username:
```

12.2 Login e invio di un messaggio

Selezionare l'opzione 1 (Login), inserire username e password. Dopo l'autenticazione, dal menu principale scegliere l'opzione 1 (Invia messaggio). Il client richiede il nome del destinatario, l'oggetto e il testo. Il server verifica che il destinatario sia un utente registrato: in caso negativo risponde con NO_RECEIVER e il messaggio non viene salvato; in caso affermativo archivia il messaggio in messages.txt e conferma con AUTH_OK.

```
1. Login
2. Registrazione
Scelta: 1
Username: alessio
Password: 123
Accesso riuscito come alessio
```

12.3 Lettura dei messaggi ricevuti

Dal menu principale selezionare l'opzione 2 (Leggi messaggi). Il client invia una richiesta READ_MESSAGES. Il server scorre messages.txt e invia al client tutti i pacchetti MESSAGE_LIST il cui campo receiver corrisponde all'utente autenticato. Per ciascun messaggio il client stampa ID, data/ora di ricezione, mittente, oggetto e testo. La ricezione termina dopo un timeout di 200 ms.

```
--- Menu ---
1. Invia messaggio
2. Leggi messaggi
3. Cancella messaggio (usando ID)
4. Esci
Scelta:
```

12.4 Cancellazione di un messaggio

Selezionare l'opzione 3 (Cancella messaggio). Il client mostra prima la lista completa dei messaggi ricevuti (con i rispettivi ID), poi richiede l'ID del messaggio da cancellare. Il server cerca il messaggio con quell'ID nel file, verificando che il destinatario corrisponda all'utente autenticato per impedire cancellazioni non autorizzate, e risponde con OK o NOT_FOUND.

13. Conclusioni

Il sistema sviluppato implementa un servizio di messaggistica completo conforme alle specifiche del progetto.

Le principali caratteristiche sono:

- architettura **client–server concorrente**
- comunicazione tramite **socket TCP**
- **autenticazione** utenti
- gestione **persistente** dei messaggi su file
- sincronizzazione tramite **mutex**
- gestione dei **segnali** e degli **errori**

Il sistema risulta scalabile e permette a più client di utilizzare simultaneamente il servizio di messaggistica.